Program Name	**B.C.A**	Semester	**VI**
Course Title	**Fundamentals of Data Science (Theory)**		
Course Code:	**DSE-E2**	No. of Credits	**03**
Contact hours	**42 Hours**	Duration of SEA/Exam	**2 1/2 Hours**
Formative Assessment Marks	**40**	Summative Assessment Marks	**60**

Course Outcomes (COs): After the successful completion of the course, the student will be able to:

CO1	Understand the concepts of data and pre-processing of data.
CO2	Know simple pattern recognition methods
CO3	Understand the basic concepts of Clustering and Classification
CO4	Know the recent trends in Data Science

Contents	**42 Hrs**
Unit I: Data Mining: Introduction, Data Mining Definitions, Knowledge Discovery in Databases (KDD) Vs Data Mining, DBMS Vs Data Mining, DM techniques, Problems,Issues and Challenges in DM, DM applications.	**8**
Data Warehouse: Introduction, Definition, Multidimensional Data Model, Data Cleaning, Data Integration and transformation, Data reduction, Discretization	**8**
Mining Frequent Patterns: Basic Concept – Frequent Item Set Mining Methods - Aprioriand Frequent Pattern Growth (FPGrowth) algorithms -Mining Association Rules	**8**
Classification: Basic Concepts, Issues, Algorithms: Decision Tree Induction. Bayes Classification Methods, Rule-Based Classification, Lazy Learners (or Learning from yourNeighbors), k Nearest Neighbor. Prediction - Accuracy- Precision and Recall	**10**
Clustering: Cluster Analysis, Partitioning Methods, Hierarchical Methods, Density-Based Methods, Grid-Based Methods, Evaluation of Clustering	**8**

Unit 4

Topics:

Classification: Basic Concepts, Issues, Algorithms: Decision Tree Induction. Bayes Classification Methods, Rule-Based Classification, Lazy Learners (or Learning from your Neighbors), k Nearest Neighbor. Prediction - Accuracy- Precision and Recall.

Classification Basic Concepts

Def: Classification is a supervised machine learning method where the model tries to predict the correct label of a given input data.

Classification is a two step process,

1. Learning/Training Step:

Here a model is constructed for classification. A classifier model is built by analyzing the data which are labeled already. Because the class label of each training tuple is provided, this step is also known as supervised learning. This stage can also be viewed as a function, y=f(x), that can predict the associated class label 'y' of a given tuple x considering attribute values. This mapping function is represented in the form of classification rules, decision trees or mathematical formula.

2. Classification/Testing Step:

Here the model that is constructed in the learning step is used to predict class labels for given data.

The data used in learning step is called "Training data".

The data used in classification step is called "Testing data".

Ex: A bank loans officer needs to clarify the loan applicant as safe or risky Figure 8.1

The accuracy of classifier on a given test set is the percentage of test set tuples that are correctly classified by the classifier.

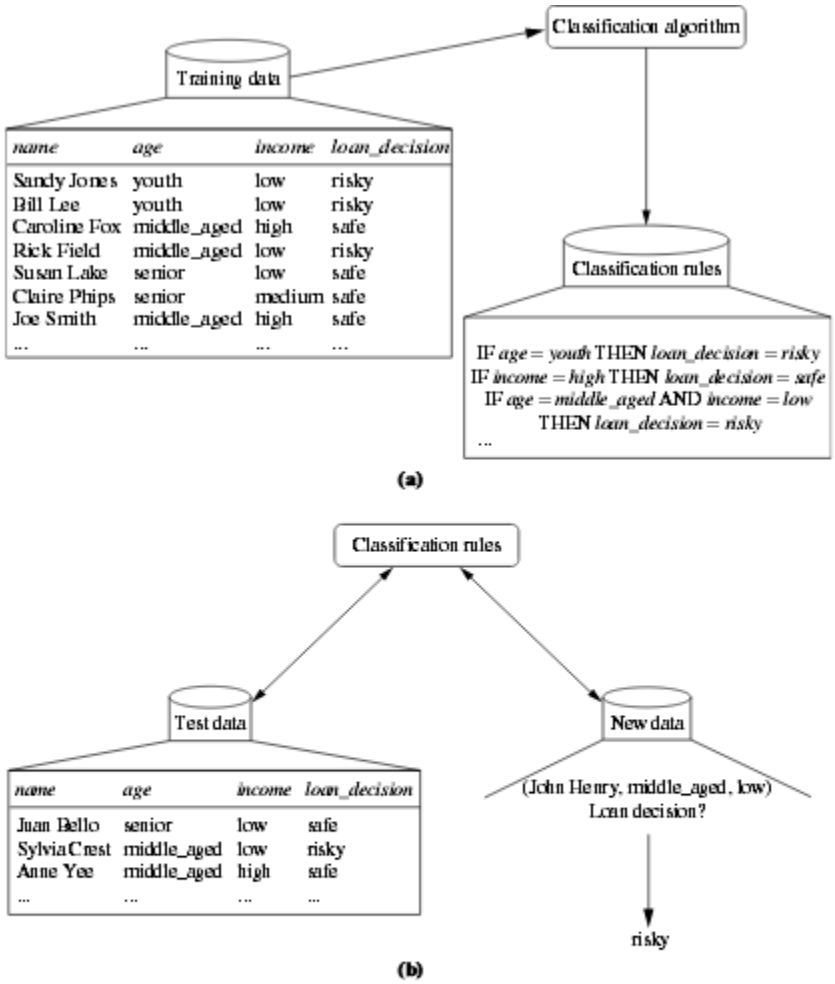


Figure 8.1 The data classification process: (a) *Learning*: Training data are analyzed by a classification algorithm. Here, the class label attribute is *loan_decision*, and the learned model or classifier is represented in the form of classification rules. (b) *Classification*: Test data are used to estimate the accuracy of the classification rules. If the accuracy is considered acceptable, the rules can be applied to the classification of new data tuples.

Decision Tree Induction:

History:

- Developed by J Ross Quinlan around 1980's
- Named it as ID3 (Iterative Dichotomiser)
- Another variant C4.5(successor of ID3)
- Another work CART(Classification And Regression Trees)
- Adopted greedy approach in which decision trees are constructed in a top-down recursive divide and conquer method

Decision Tree Induction is the learning of decision trees from class-labelled training tuples. Decision Tree is a tree structure, where each internal node (non leaf) denotes a test on the attribute, each branch represents an outcome of the test and each leaf node holds a class label. The attribute values of a tuple 'X' is tested against the decision tree. A path is traced from the root to leaf to predict the class label.

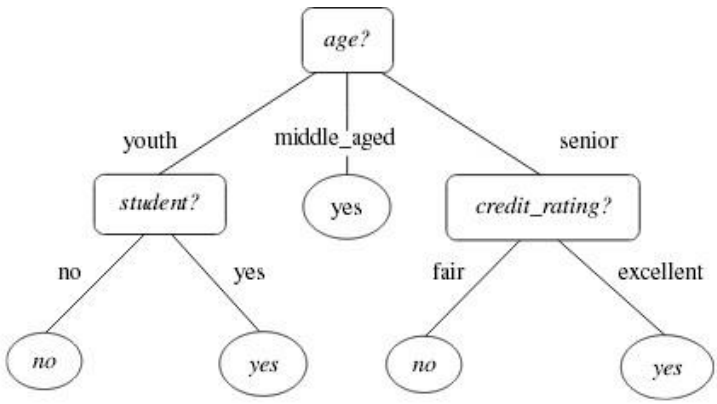


Figure 8.2 A decision tree for the concept *buys_computer*, indicating whether an *AllElectronics* customer is likely to purchase a computer. Each internal (nonleaf) node represents a test on an attribute. Each leaf node represents a class (either *buys_computer* = *yes* or *buys_computer* = *no*).

Advantages:

- DT can be easily converted into classification ruler(if---then)
- DT does not require any domain knowledge and hence easy to construct and interpret
- DT can handle multidimensional data
- DT is simple and fast

Applications:

Medicine, manufacturing and production, finance, astronomy, molecular biology.

Algorithm: Decision Tree Generation

Input: D = set of training tuples with associated class labels.

Attribute list= attributes of tuple.

Attribute_selection_method= a function to determine the splitting criteria that best partitions the data tuples into individual classes.

Output: A Decision Tree

Method:

Create a node N;

If tuples in D are all of the same class, C then return N as a leaf node labeled with the class C;

If attribute list is empty then

Return N as the leaf node labeled with the majority class in D;

Apply attribute selection method(D, attribute list) to find the best splitting criterion;

Label node 'N' with splitting criterion;

If splitting attribute is discrete valued and multiway splits allowed then

attribute list ← attribute list – splitting attribute;//remove split attribute.

For each outcome 'j' of splitting criterion// partition tuple and sub trees.

Let D_j be the set of data tuples in D satisfying outcome j;

If D_j is empty then

Attach a leaf labeled with majority class in D to node N;

Else

Attach the node returned by decision tree generation (D_j, attribute list) to node N

End for

Return N;

Splitting Scenarios:

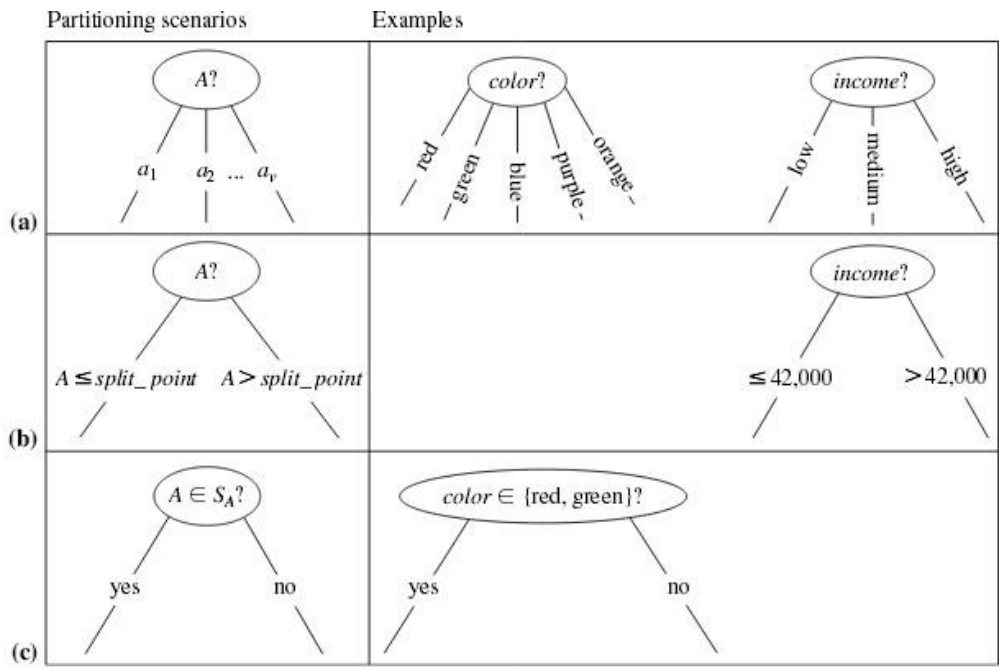


Figure 8.4 This figure shows three possibilities for partitioning tuples based on the splitting criterion, each with examples. Let A be the splitting attribute. (a) If A is discrete-valued, then one branch is grown for each known value of A. (b) If A is continuous-valued, then two branches are grown, corresponding to $A \leq split_point$ and $A > split_point$. (c) If A is discrete-valued and a binary tree must be produced, then the test is of the form $A \in S_A$, where S_A is the splitting subset for A.

Attribute Selection Measures:

It is a measure for selecting the splitting criterion that 'best' separates a given data partition D of class labeled training tuples into individual classes. It is also called as splitting rules.

The attribute having the best measure is chosen as the splitting attribute for the given tuples.

Popular Attribute Selection Measures:

1. Information gain
2. Gain ratio
3. Gini index

Information Gain:

Based on the work by Claude Shanon on information theory. Information Gain is defined as the difference between the original information requirement and new requirement. The attribute with highest information gain is chosen as the splitting attribute for node N. It is used in ID3.

Gain(A) = Info(D) – $Info_A(D)$

Where,

$$\textbf{Info(D)} = \sum_{i=0}^{m} Pi\ log_2(Pi)$$

m = m distinct classes

Pi = probability that a tuple belongs to class C_i and estimated using

---

Info(D) is also known as entropy of D

How much more info is needed to arrive at an exact classification is given by i.e. computing expected info gain for each attribute.

$$\textbf{Info}_A\textbf{(D)} = -\sum_{j=1}^{v} \frac{|Dj|}{|D|} * \textbf{Info(D}_j\textbf{)}$$

Refer eg: 8.1

Table 8.1 Class-Labeled Training Tuples from the *AllElectronics* Customer Database

RID	*age*	*income*	*student*	*credit_rating*	*Class: buys_computer*
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Example 8.1 Induction of a decision tree using information gain. Table 8.1 presents a training set, D, of class-labeled tuples randomly selected from the *AllElectronics* customer database. (The data are adapted from Quinlan [Qui86]. In this example, each attribute is discrete-valued. Continuous-valued attributes have been generalized.) The class label attribute, *buys_computer*, has two distinct values (namely, {*yes, no*}); therefore, there are two distinct classes (i.e., $m = 2$). Let class C_1 correspond to *yes* and class C_2 correspond to *no*. There are nine tuples of class *yes* and five tuples of class *no*. A (root) node N is created for the tuples in D. To find the splitting criterion for these tuples, we must compute the information gain of each attribute. We first use Eq. (8.1) to compute the expected information needed to classify a tuple in D:

$$Info(D) = -\frac{9}{14}\log_2\left(\frac{9}{14}\right) - \frac{5}{14}\log_2\left(\frac{5}{14}\right) = 0.940 \text{ bits.}$$

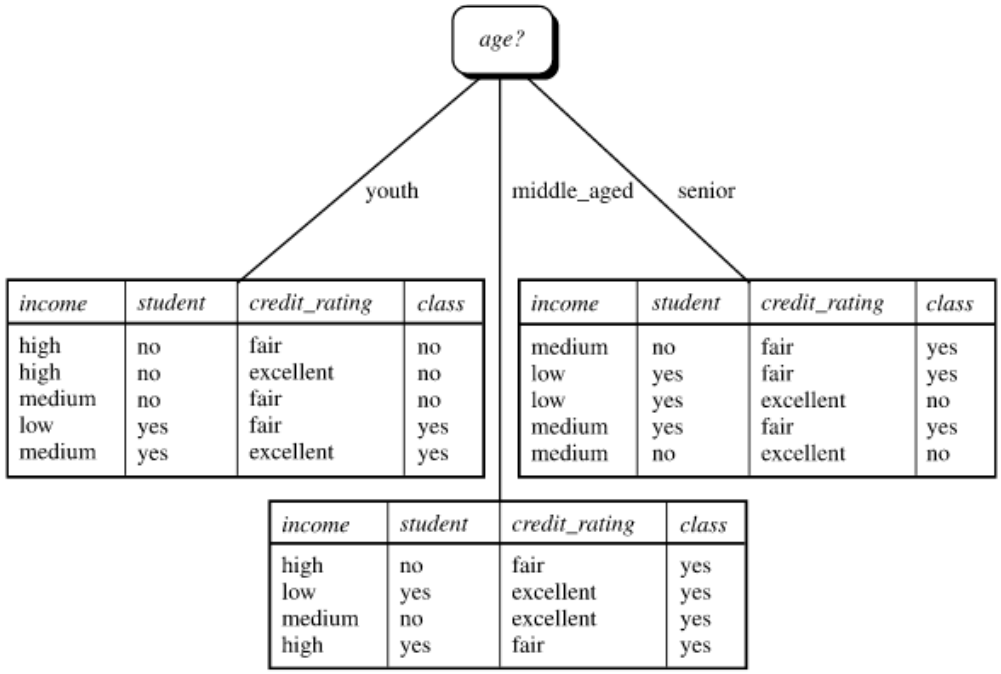
Next, we need to compute the expected information requirement for each attribute. Let's start with the attribute *age*. We need to look at the distribution of *yes* and *no* tuples for each category of *age*. For the *age* category "youth," there are two *yes* tuples and three *no* tuples. For the category "middle_aged," there are four *yes* tuples and zero *no* tuples. For the category "senior," there are three *yes* tuples and two *no* tuples. Using Eq. (8.2), the expected information needed to classify a tuple in D if the tuples are partitioned according to *age* is

$$\begin{aligned} Info_{age}(D) = {} & \frac{5}{14} \times \left(-\frac{2}{5}\log_2\frac{2}{5} - \frac{3}{5}\log_2\frac{3}{5}\right) \\ & + \frac{4}{14} \times \left(-\frac{4}{4}\log_2\frac{4}{4}\right) \\ & + \frac{5}{14} \times \left(-\frac{3}{5}\log_2\frac{3}{5} - \frac{2}{5}\log_2\frac{2}{5}\right) \\ & = 0.694 \text{ bits.} \end{aligned}$$

Hence, the gain in information from such a partitioning would be

$$Gain(age) = Info(D) - Info_{age}(D) = 0.940 - 0.694 = 0.246 \text{ bits.}$$

Similarly, we can compute $Gain(income) = 0.029$ bits, $Gain(student) = 0.151$ bits, and $Gain(credit_rating) = 0.048$ bits. Because *age* has the highest information gain among the attributes, it is selected as the splitting attribute. Node N is labeled with *age*, and branches are grown for each of the attribute's values. The tuples are then partitioned accordingly, as shown in Figure 8.5. Notice that the tuples falling into the partition for *age* = *middle_aged* all belong to the same class. Because they all belong to class "*yes*," a leaf should therefore be created at the end of this branch and labeled "*yes*." The final decision tree returned by the algorithm was shown earlier in Figure 8.2. ■



income	student	credit_rating	class
high	no	fair	no
high	no	excellent	no
medium	no	fair	no
low	yes	fair	yes
medium	yes	excellent	yes

income	student	credit_rating	class
medium	no	fair	yes
low	yes	fair	yes
low	yes	excellent	no
medium	yes	fair	yes
medium	no	excellent	no

income	student	credit_rating	class
high	no	fair	yes
low	yes	excellent	yes
medium	no	excellent	yes
high	yes	fair	yes

Figure 8.5 The attribute *age* has the highest information gain and therefore becomes the splitting attribute at the root node of the decision tree. Branches are grown for each outcome of *age*.

Gain Ratio:

Information gain measure is biased toward test with many outcomes (many partitions). That is, it prefers to select attributes having a large number of values. For example, consider an attribute that acts as a unique identifier such as product ID. Split on product ID would result in a large number of partitions (as many as there are values), each one containing just one tuple. Gain ratio overcomes this bias. Attribute with maximum gain ratio is selected on splitting attribute. IT is used in C4.5

Gain ratio is given by the following formula:

$$\textbf{Gain ratio(A)} = \frac{\mathbf{Gain(A)}}{\mathbf{SplitInfoA(D)}}$$

It applies a kind of normalization to information gain using a split information value defined as

$$\textbf{SplitInfo}_{\mathbf{A}}\textbf{(D)} = -\sum_{j=1}^{v} \frac{|Dj|}{|D|} * \log_2 \frac{|Dj|}{|D|}$$

v = partitions

Example 8.2 **Computation of gain ratio for the attribute *income*.** A test on *income* splits the data of Table 8.1 into three partitions, namely *low*, *medium*, and *high*, containing four, six, and four tuples, respectively. To compute the gain ratio of *income*, we first use Eq. (8.5) to obtain

$$SplitInfo_{income}(D) = -\frac{4}{14} \times \log_2\left(\frac{4}{14}\right) - \frac{6}{14} \times \log_2\left(\frac{6}{14}\right) - \frac{4}{14} \times \log_2\left(\frac{4}{14}\right)$$

$$= 1.557.$$

From Example 8.1, we have $Gain(income) = 0.029$. Therefore, $GainRatio(income) = 0.029/1.557 = 0.019$. ■

Gini Index:

It measures the impurity/uncertainty/randomness of Dataset D. It is given by the following formula. It is used in CART.

$$\textbf{Gini(D)} = 1 - \sum_{i=1}^{m} Pi^2$$

D_i = probability of tuple in D belong to C_i; **P_i** = $\frac{|C_i,D|}{|D|}$; **m=** classes.

Tree Pruning:

When decision tree is built, many of the branches will reflect problems in the training data due to noise or outliers. Tree pruning removes the branches which are not relevant. Pruned tree are smaller and less complex and thus easier to understand. They perform faster than unpruned trees.

There are two common approaches for tree pruning:

- **Pre pruning:** In pre pruning the tree branch is not further split into sub branches by deciding early using statistical measures like info gain, gini index etc.
- **Post pruning:** In post pruning the fully grown tree branches is cut and leaf nodes are added. The leaf is labeled with the most frequent class among the subtree being replaced.

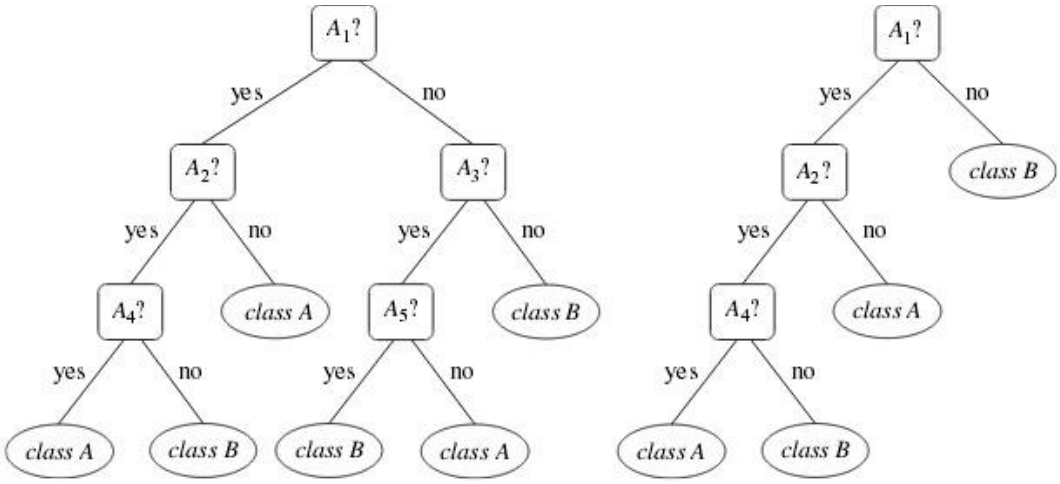


Figure 8.6 An unpruned decision tree and a pruned version of it.

Pruning Algorithms:

- Cost complexity pruning algorithm used in CART.
- Pessimistic pruning used in C4.5.

Disadvantages of decision trees:

- Suffers from repetition (same node repeats in the tree) and replication(redundant sub trees).

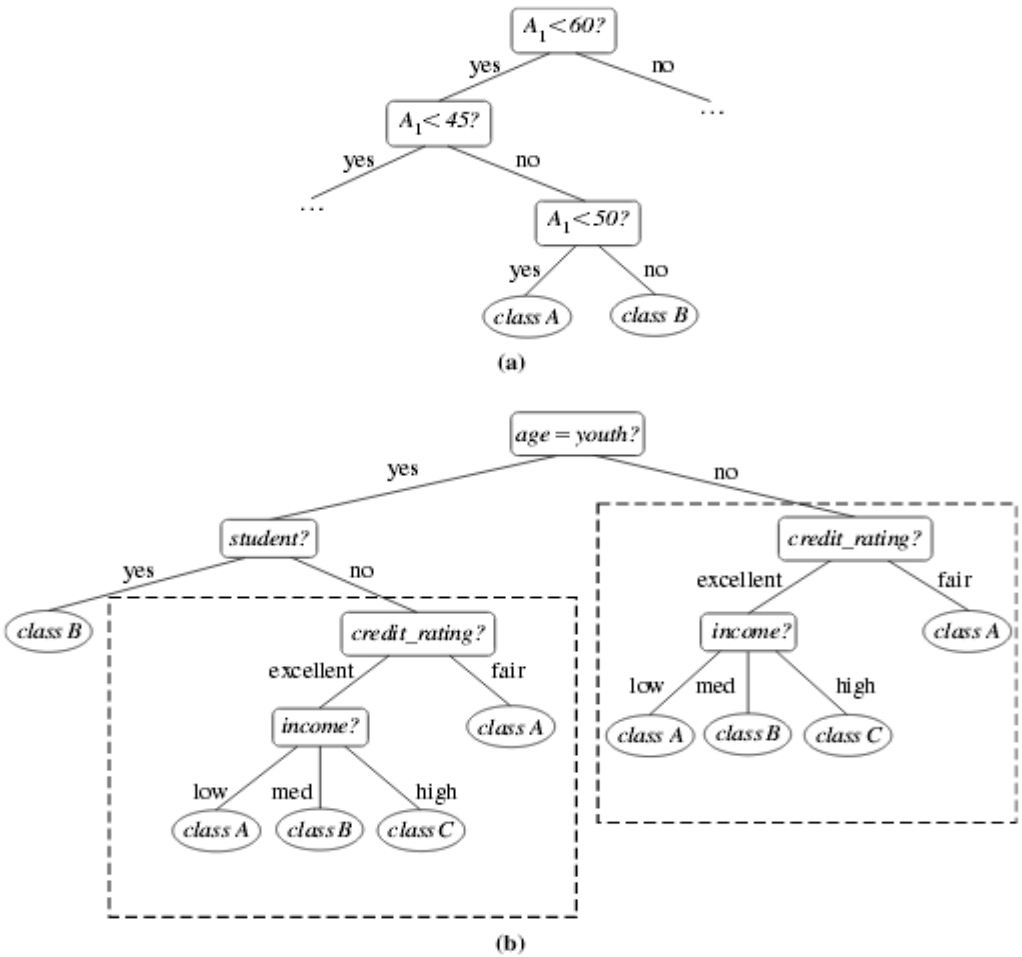


Figure 8.7 An example of: (a) subtree **repetition**, where an attribute is repeatedly tested along a given branch of the tree (e.g., *age*) and (b) subtree **replication**, where duplicate subtrees exist within a tree (e.g., the subtree headed by the node "*credit_rating?*").

-
- Use of multivariate split reduces the above problem.

Bayes Classification Method

- Bayes Classification (BC) are statistical classifiers.
- B C are based on Bayes Theorem
- B C predict class membership probabilities such as the probability that a given tuple belongs to a particular class
- A simple B C is called as Naive B C.
- N B C assumes that the effect of an attribute value on a given class is independent of the values of other attributes. This assumption is called class-conditional independence

  Example for conditional Independence:

  A fruit may be considered to be an apple if it is red, round and 3" diameter. A Naïve Bayes Classifier considered each of these features to contribute independently to the

probability that this fruit is an apple, regardless of any possible correlations between the color, roundness and diameter.

Review of Bayes Theorem:

Named after Thomas Bayes of 18th century.

Bayes Theorem is:

$$P(H/X) = \frac{P(\frac{X}{H})\, P(H)}{P(X)}$$

H – Hypothesis that belongs to a particular class

X – Data type(measurements on n-attributes)

P(H/X) – Posterior probability, i.e. probability that X belong to hypothesis made on H

Ex: Probability that a customer X will buy a computer given that we know the age and income of customer.

P(X/H) - Posterior probability, i.e. probability that X belong to hypothesis made on H

Ex: Probability that a customer X (Rs.40,000) will buy a computer given that we know the customer will buy a computer.

P(H) – Prior probability of H

Ex: Probability that any given customer will buy a computer regardless of measurements on attribute.

Naïve Bayesian Classification:

N B classifiers works as follows:

1) Let D be a training set of tuples(with class labels)
   X – tuple with n-attributes.
2) Suppose that there are m classes C1, C 2,….., Cm. Given a tuple X, the classifier will predict that X belong to the class having the highest posterior probability conditioned on X.
   i.e.
   $P(C_i/X) > P(C_j/X)$ for $1 \leq j \leq m$, $j \neq i$.
   Thus we maximize $P(C_i/X)$. The class C_i for which $P(C_i/X)$ is maximized is called the maximum posterior hypothesis.
   By Bayes Theorem

$$P(C_i/X) = \frac{P(\frac{X}{Ci})\, P(Ci)}{P(X)}$$

3) Since P(X) is constant for all classes, therefore denominator is not considered and only $P(X/C_i)\ P(C_i)$ needs to be computed for all the classes.
4) To predict the class label of X, P(X/Ci) P(Ci) is evaluated for all class C and maximum of P(X/Ci) P(Ci) is assigned as class label.

Example: Classifying if we can play based on weather

If we can play on a sunny day?

Dataset (Training)

Weather	**Play**
Sunny	Yes
Over cast	No
Rainy	No
Sunny	No
Over cast	Yes
Rainy	No
Sunny	Yes
Over cast	No

i.e. P(Yes/Sunny) > P(No/Sunny)

$$P(\text{Yes/Sunny}) = \frac{P(\frac{\text{Sunny}}{\text{Yes}})P(\text{Yes})}{P(\text{Sunny})} = \left[\frac{\binom{2}{3}*\binom{3}{8}}{\frac{3}{8}}\right] = (0.66*0.37)/0.37=0.66$$

$$P(\text{No/sunny}) = \frac{P(\frac{\text{Sunny}}{\text{No}})P(\text{No})}{P(\text{Sunny})} = \left[\frac{\binom{1}{3}*\binom{5}{8}}{\frac{3}{8}}\right] = (0.33*0.62)/0.37=0.55$$

Since P(Yes/sunny > P(No/sunny) , based on Bayes Classification We can play on a sunny day.

Advantages

- Easy to implement
- Good results are obtained in most of the cases

Disadvantages

- Assumption: class conditional independence, therefore loss of accuracy
- Practically, dependencies exist among variables
    - E.g., hospitals: patients: Profile: age, family history, etc.

Symptoms: fever, cough etc., Disease: lung cancer, diabetes, etc.

- Dependencies among these cannot be modeled by Naïve Bayes Classifier

Rule Based Classifiers

Rule-based classifiers are used for classification by defining a set of rules that can be used to assign class labels to new instances of data based on their attribute values. These rules can be created using expert knowledge of the domain, or they can be learned automatically from a set of labeled training data. A rule-based classifier uses a set of IF-THEN rules for classification.

An IF-THEN rule is an expression of the form:

IF condition THEN conclusion.

An example is rule R1,

R1: IF age == youth AND student == yes THEN buys computer == yes.

The “IF” part (or left side) of a rule is known as the rule antecedent or precondition.

The “THEN” part (or right side) is the rule consequent.

In the rule antecedent, the condition consists of one or more attribute tests (e.g., age == youth and student == yes). that are logically ANDed.

The rule’s consequent contains a class prediction (in this case, we are predicting whether a customer will buy a computer).

R1 can also be written as R1: (age D youth) ^ (student D yes))(buys computer D yes).

If the condition (i.e., all the attribute tests) in a rule antecedent holds true for a given tuple, we say that the rule antecedent is satisfied (or simply, that the rule is satisfied) and that the rule covers the tuple.

If more than one rule applies for a given tuple then conflict arises on which rule to be selected. Conflict resolution strategies such as rule ordering and size ordering must be applied to break the tie or conflict.

In rule-based ordering, the rules are organized based on priority, according to some measure of rule quality, such as accuracy, coverage, or size (number of attribute tests in the rule antecedent), or based on advice from domain experts. When rule ordering is used, the rule set is known as a decision list. Class is predicted for the tuple based on the priority, and any other rule that satisfies tuple is ignored.

In size-based ordering, the rule with the toughest requirements, where the toughest means the longest list of conditions satisfied in the antecedent part of the rule (e.g., 3 conditions out of 5 are satisfied) will be chosen and tie is broken.

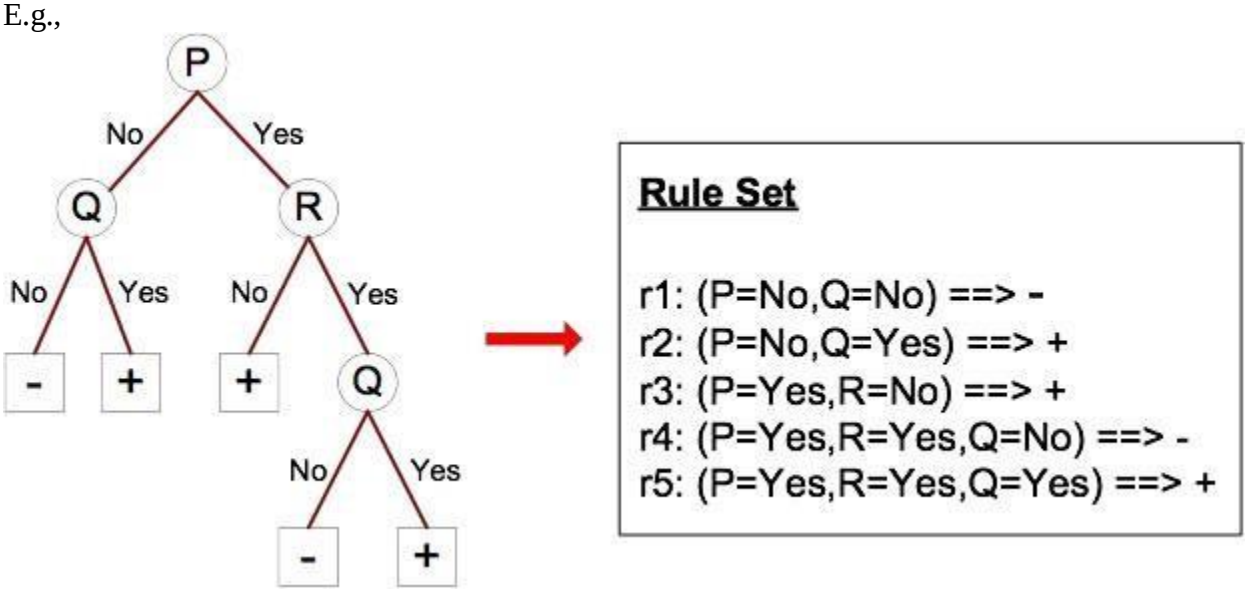
Coverage and Accuracy: Measures to evaluate the accuracy of the rules.

In rule-based classification, coverage is the percentage of records that satisfy the antecedent conditions of a rule.
Coverage(R)=n1/n
Where n1= instances with antecedent and n=no of training tuples

Accuracy is the percentage of records that satisfy the antecedent conditions and meet the consequent values of a rule.

Accuracy(R)=n2/n1
Where n2= instances with antecedent AND consequent

Rule Extraction from a Decision Tree
Use the decision tree and generate the rules from each branch. Where nodes will become antecedent and leaf node will be consequent. Logical AND is used to combine the nodes of the tree.
E.g.,



Rule Induction Using a Sequential Covering Algorithm
Sequential Covering is a popular algorithm based on Rule-Based Classification. IF-THEN rules can be extracted directly from the training data (i.e., without having to generate a decision tree first) using a sequential covering algorithm.
There are many sequential covering algorithms. Popular variations include AQ, CN2, and the more recent RIPPER. The general strategy is as follows. Rules are learned one at a time. Each time a rule is learned, the tuples covered by the rule are removed, and the process repeats on the remaining tuples.
Algorithm: Sequential covering. Learn a set of IF-THEN rules for classification.
Input:

D: a data set of class-labeled tuples;
Att vals: the set of all attributes and their possible values.

Output: A set of IF-THEN rules.
Method:
(1) Rule set = {}; // initial set of rules learned is empty
(2) for each class c do
(3) repeat
    (4) Rule = Learn One Rule(D, Att vals, c);
    (5) remove tuples covered by Rule from D;
    (6) Rule set = Rule set + Rule; // add new rule to rule set
(7) until terminating condition;
(8) endfor
(9) return Rule Set ;

Rule Pruning
The rule is pruned is due to the following reason −

The Assessment of quality is made on the original set of training data. The rule may perform well on training data but less well on subsequent data. That's why the rule pruning is required.

The rule is pruned by removing conjunct. The rule R is pruned, if pruned version of R has greater quality than what was assessed on an independent set of tuples.

FOIL is one of the simple and effective method for rule pruning. For a given rule R,

FOIL_Prune = pos - neg / pos + neg
where pos and neg is the number of positive tuples covered by R, respectively.

Note − This value will increase with the accuracy of R on the pruning set. Hence, if the FOIL_Prune value is higher for the pruned version of R, then we prune R.

Advantages of Rule-Based Classifiers
- Has characteristics quite similar to decision trees
- As highly expressive as decision trees
- Easy to interpret
- Faster Performance comparable to decision trees
- Can handle redundant attributes

Lazy Learners: K-Nearest Neighbor Classifier Algorithm

The classification methods discussed such as decision tree induction, Bayesian classification, rule-based classification, etc., are all examples of *eager learners*. Eager learners employ two step approach for classification, i.e., in first step they build classifier model learning from the training set and in second step they perform the classification on unknow tuples to know class using the model.

Lazy learning algorithms wait until they encounter a new tuple(From testing dataset), then store and compare training examples when making predictions. This type of learning is useful when working with large datasets that have a few attributes. Lazy learning is also known as instance-based or memory-based learning.

Examples of lazy learners: K-Nearest Neighbor Classifier, case-based reasoning classifiers.

Advantages of Lazy learners:

- can adapt quickly to new or changing data.
- less affected by outliers compared to eager learning methods.
- Handles complex data distributions and nonlinear relationships

Disadvantages of lazy learners:

- Computationally expensive
- Required more memory as training data will be loaded only during classification stage.

K-Nearest Neighbor Classifier Algorithm

- The *k*-nearest-neighbour method was first described in the early 1950s.
- widely used in the area of pattern recognition.
- The K-Nearest Neighbors (KNN) algorithm is a popular machine learning technique used for classification and regression tasks. It relies on the idea that similar data points tend to have similar labels or values.

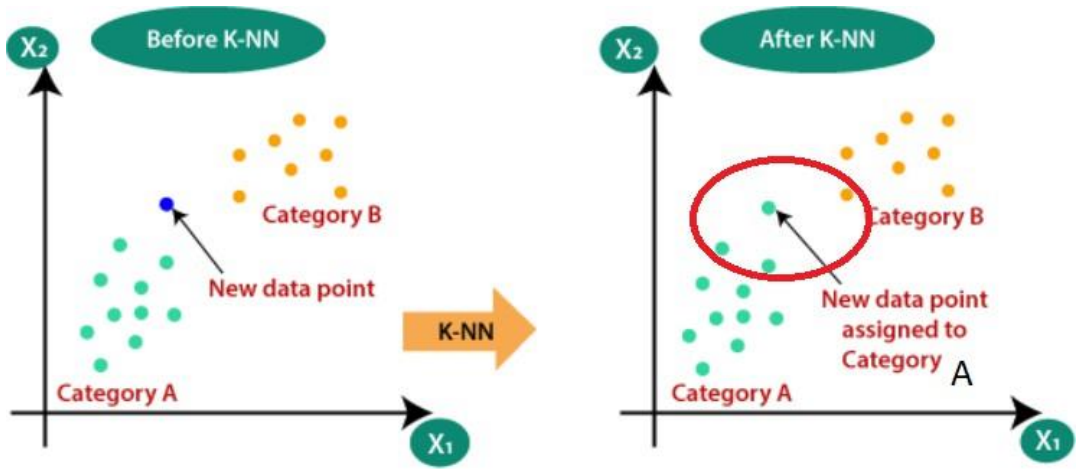
The steps for the KNN algorithm are:

1. Assign a value to K
2. Calculate the distance(E.g, Euclidean Distance) between the new data entry and all other existing data entries
3. Arrange the distances in ascending order
4. Determine the k-closest records of the training data set for each new record
5. Take the majority vote to classify the data point.

The Euclidean distance between two points or tuples, say, X1 ={x11, x12, : : : , x1n} and X2 ={x21, x22, : : : , x2n}, is

$$dist(\boldsymbol{X}_1, \boldsymbol{X}_2) = \sqrt{\sum_{i=1}^{n} (x_{1i} - x_{2i})^2}.$$

Example:



Advantages

- Simple and efficient
- Easy to implement
- No assumption required
- No training time required

Disadvantages:

- Computationally expensive
- Accuracy reduces if there are noise in the dataset
- Requires large memory
- Need to accurately determine the value of k neighbors

Metrics for Evaluating Classifier Performance (Precision and Recall)

There are four terms we need to know that are the "building blocks" used in computing many evaluation measures. Understanding them will make it easy to grasp the meaning of the various measures.

True positives (TP): These refer to the positive tuples that were correctly labeled by the classifier. E.g., Person with COVID-19 correctly labelled as COIVD-19 positive.

True negatives (TN): These are the negative tuples that were correctly labeled by the classifier.

E.g., Person without having COVID-19 virus is correctly labelled as COIVD-19 negative.

False positives (FP): These are the negative tuples that were incorrectly labeled as positive.

E.g., Person without having COVID-19 virus is incorrectly labelled as COIVD-19 positive.

False negatives (FN): These are the positive tuples that were mislabeled as negative.

E.g., Person having COVID-19 virus is incorrectly labelled as COIVD-19 negative.

Precision and Recall:

Precision and recall are metrics used to evaluate the performance of classification models in machine learning. Precision is the percentage of positive identifications that are correct, while recall is the percentage of actual positives that are identified correctly.

$$precision = \frac{TP}{TP + FP}$$

$$recall = \frac{TP}{TP + FN}$$

Here's an example of precision and recall:

- If you have 20 items and the model predicted 10 items, out of these, 4 predictions were wrong and 6 were correct.
- The precision is (6/10), ie 60% while the recall is (6/20) ie 30%

Accuracy:

The accuracy of a classifier on a given test set is the percentage of test set tuples that are correctly classified by the classifier. In the pattern recognition literature, this is also referred to as the overall recognition rate of the classifier, that is, it reflects how well the classifier recognizes tuples of the various classes. That is,

$$accuracy = \frac{TP + TN}{P + N}.$$

The sensitivity and specificity measures can be used, respectively, for this purpose. Sensitivity is also referred to as the true positive (recognition) rate (i.e., the proportion of positive tuples that are correctly identified), while specificity is the true negative rate (i.e., the proportion of negative tuples that are correctly identified). These measures are defined as

$$sensitivity = \frac{TP}{P}$$

$$specificity = \frac{TN}{N}.$$

It can be shown that accuracy is a function of sensitivity and specificity:

$$accuracy = sensitivity\frac{P}{(P+N)} + specificity\frac{N}{(P+N)}.$$

Confusion Matrix:

A confusion matrix represents the prediction summary in matrix form. It shows how many prediction are correct and incorrect per class.

		Predicted class		
		yes	*no*	Total
Actual class	*yes*	*TP*	*FN*	*P*
	no	*FP*	*TN*	*N*
	Total	P'	N'	$P+N$

Confusion matrix for binary classification.

		Predicted	
		Dog	**CAT**
Actual	Dog	5	1
	CAT	2	4

Total testing samples: 12